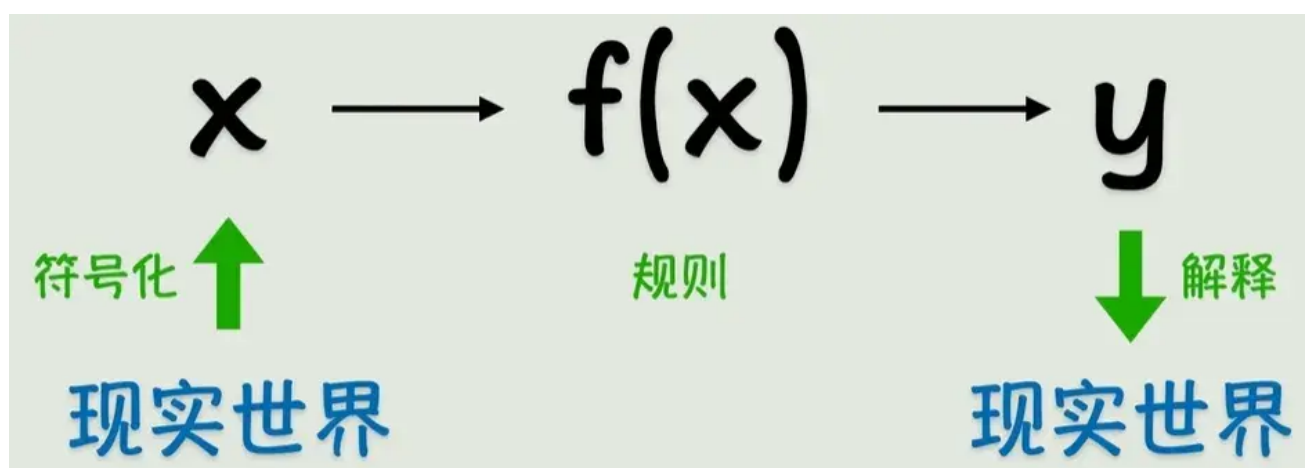


闪客发了新的合集视频，所以我把之前每p的笔记搬过来合在一起，看起来方便一点。

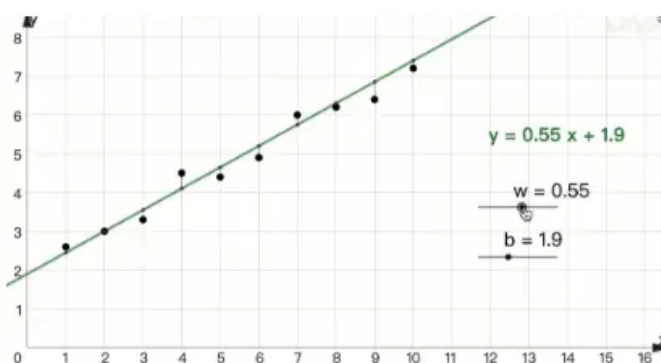
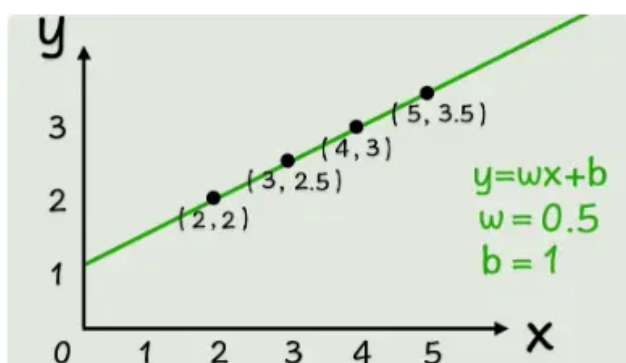
内容包含合集前6个视频的笔记记录，其实也就是重复了一遍视频内容，然后文本化了。

从函数到神经网络

事实上函数就是一种变换，对数据进行变换得到我们所需要的结果。



早期的人工智能->**符号主义**，即用精确的函数来表示一切。但是很多时候，我们没办法找到一个精确的函数来描述某个关系，退而求其次选择一个近似解也不错，也就是说函数没必要精确的通过每一个点，它只需要最接近结果就好了，这就是**联结主义**。



但是当数据稍微变化一下，出现曲线的时候，简单的线性函数就没办法解决这个问题了。那我们的目标就是将线性函数转为非线性函数，这可以通过套一个非线性函数来做到，比如平方、正弦函数、指数函数。这就是**激活函数**。

$$f(x) = wx + b \quad f(x) = \sin(wx + b)$$

$$f(x) = (wx + b)^2 \quad f(x) = e^{wx + b}$$

$$f(x) = g(wx + b)$$

激活函数

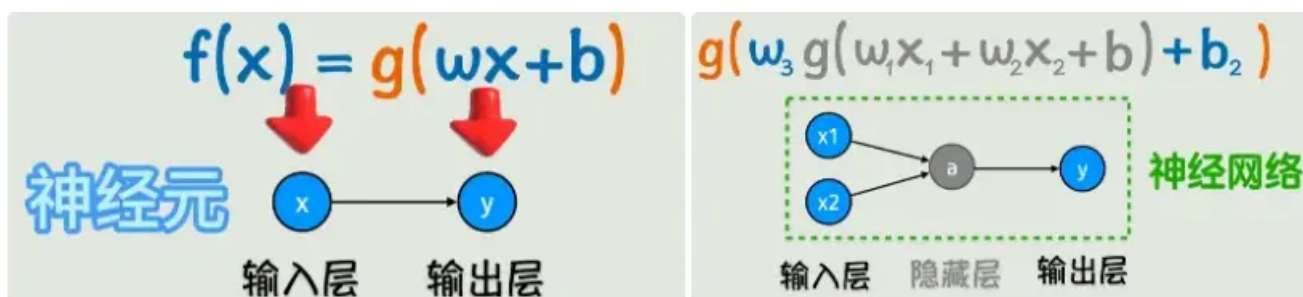
$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{ReLU}(z) = \max(0, z)$$

这还不够，正常情况下我们不会只有一个输入，并且只有一个激活函数可能不会达到理想的结果，所以开始嵌套！

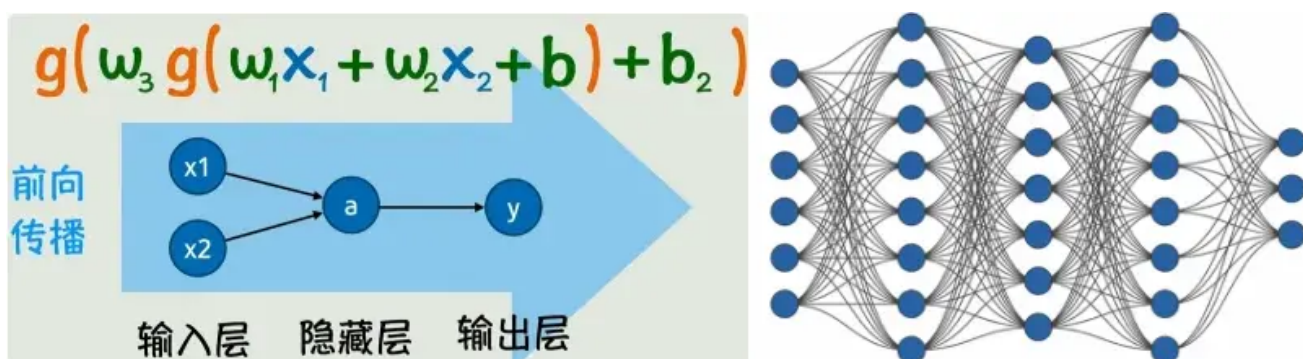
$$g(w_3 g(w_1 x_1 + w_2 x_2 + b) + b_2)$$

像这样多个输入，激活函数外面加一次线性变换，再套一个激活函数，并且还可以不断嵌套。通过这样的方式，我们可以构造出非常复杂的关系，理论上可以逼近任意的连续函数。

但这样看起来太复杂了，所以我们引入神经元的概念，图中的一个圈就是一个神经元，多个神经元互相连接形成的网状结构就叫做**神经网络**。同时我们可以看到随着函数式子的嵌套，神经网络也在拓展，原本的输出成为了**隐藏层**：位于输入层和输出层之间的中间层。隐藏层的主要作用是对输入数据进行复杂的特征提取和变换，

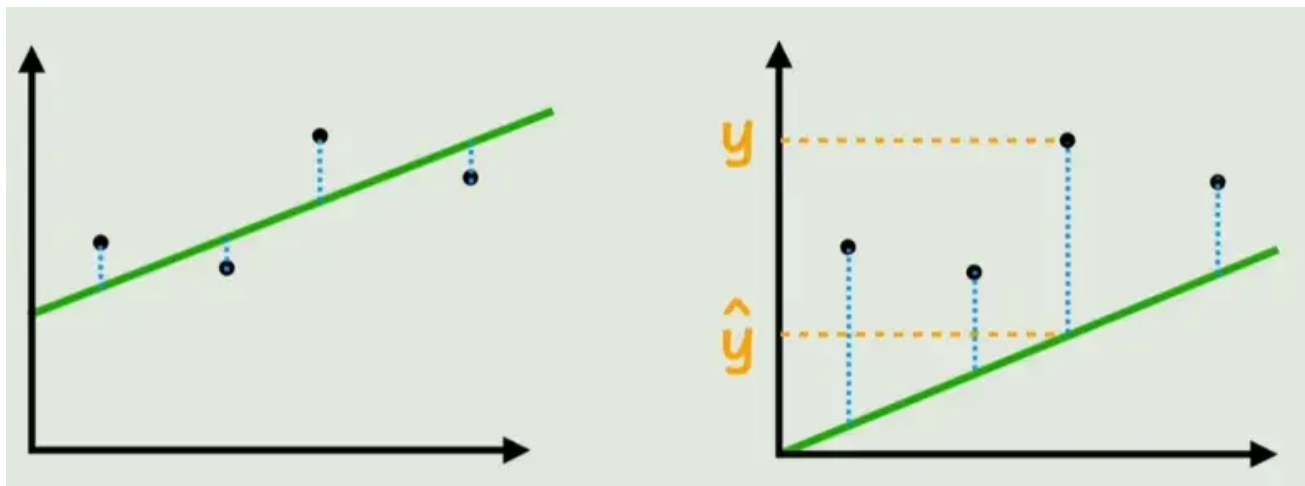


从神经网络的图示来看，就像是一个信号从左向右传播的过程，这个过程就叫做神经网络的前向传播。并且神经网络的层数和每一层的神经元都可以叠加，这样就可以构成一个非常复杂的非线性函数。看起来复杂，但是我们的目标还是一开始说的：找到一个近似解，也就是根据已知的x、y猜出所有的w和b。



如何计算神经网络的参数

那么如何计算 w 和 b 呢？首先明确我们需要的 w 和 b 能够让函数的结果更接近真实数据。对于以下数据，我们可以看出显然第一个拟合的更好。



用减去，再套上绝对值，就可以表示一个点的预测数据和真实值的误差。为了评估整体的拟合效果，我们将这些线段的长度相加，这样就得到了预测数据与真实数据的总的差异。这个函数就叫做**损失函数**：表示预测数据与真实数据误差的函数。对图中的函数进行改造，去掉绝对值进行平方，再根据样本数量进行平均化，就得到了“均方误差”（损失函数的一种）。把损失函数记为 L ，从参数的视角看， L 就是一个关于 w 、 b 的函数。

损失函数

$$\sum_{i=1}^N |y_i - \hat{y}_i|$$

均方误差 (MSE)

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

损失函数表示的是预测值与真实值的误差，而我们的目标就是让误差最小，也就是可以让损失函数 L 最小的 w 和 b 。

参数很少的时候或许可以用“令偏导数等于0”来求得损失函数最小时的 w 和 b 。通过寻找一种线性函数来拟合 x 和 y 的关系，就是机器学习中最基本的一种分析法：线性回归。

但神经网络通常是非常复杂的非线性函数，它的损失函数一般也非常复杂。这时候我们采用的办法为：**梯度下降**

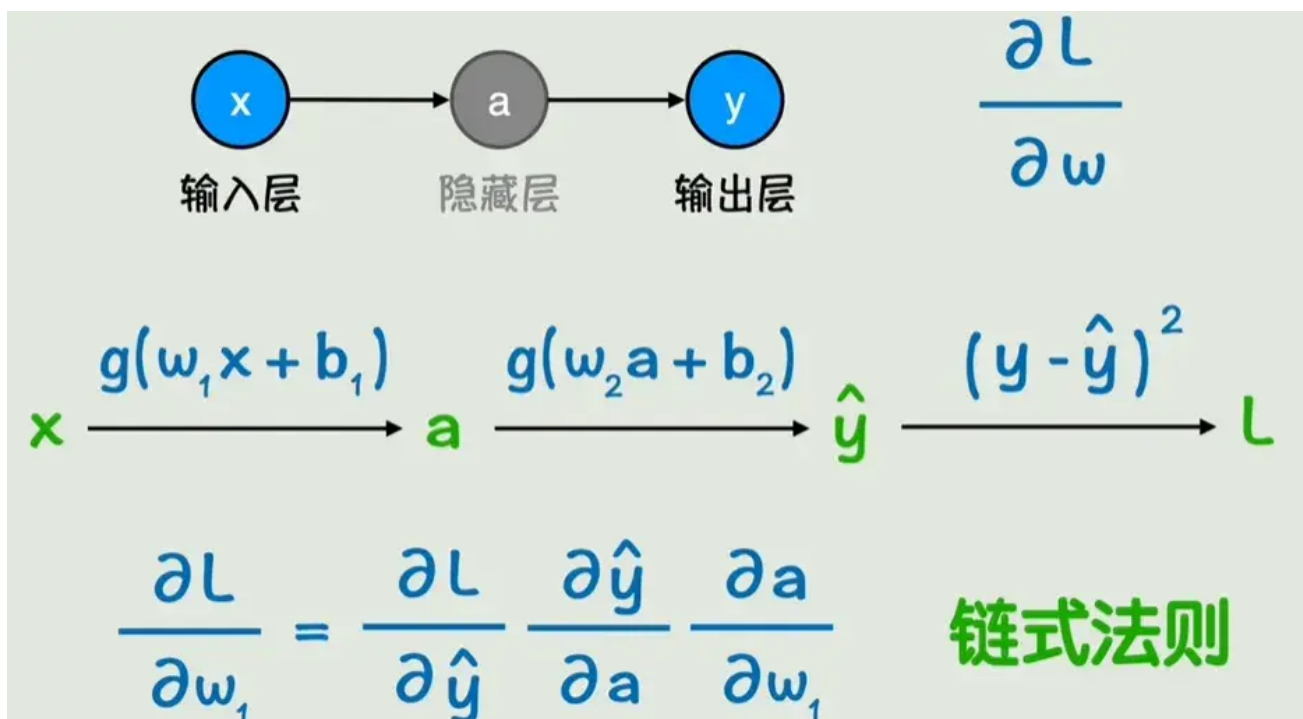
调整 w 的值（增大），如果损失函数增大，我们就反向调整 w 。就这样不断调整 w 和 b （所有参数）使得损失函数不断减小。而损失函数 L 随着参数 w 变化而变化的程度，其实就是损失函数对 w 的偏导数。而我们要做的，就是让 w 和 b 不断地向偏导数地反方向去变化。具体变化的快慢，我们再增加一个系数（**学习率**）来控制。这些偏导数所构成的向量就叫做**梯度**。不断变化 w 和 b 使得损失函数不断减小，进而求出最后的 w 和 b ，这个过程就叫做梯度下降。

$$w = w - \eta \frac{\partial L(w, b)}{\partial w}$$

$$b = b - \eta \frac{\partial L(w, b)}{\partial b}$$

梯度

现在我们的问题就变成了如何求偏导数了。用一个简单的例子来说明。要求L对w1的偏导，只需要用链式法则分别求图中的三个偏导，再相乘就好了。而由于我们可以从右向左计算这些偏导数，然后调整每一层的参数，计算前一层的时候用到的偏导数的值，后面也会用到（更左边的层），所以可以让这些值从右向左传播，这个过程就叫做**反向传播**。



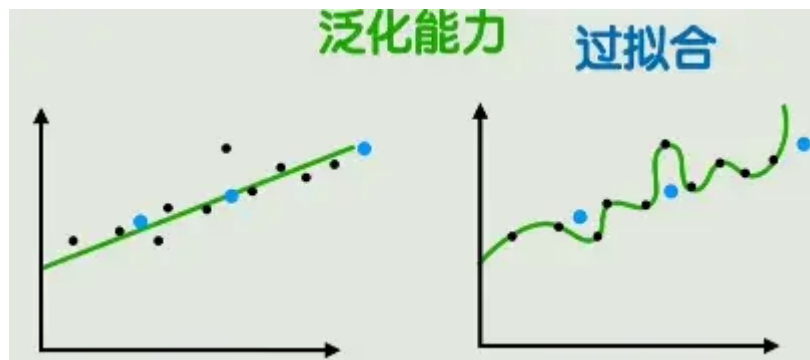
小结：我们通过前向传播根据输入x计算输出y，再根据反向传播计算损失函数关于每个参数的梯度，每个参数都向梯度的反方向变化一点，这个就是神经网络的一次训练。经过多轮训练使得损失函数足够小，就得到了我们想要的函数。

一些问题和解决方法

现在假设我们用训练数据成功训练了一个神经网络，还有可能出现哪些问题呢？

过拟合

所谓过拟合，就是模型太复杂了，在学习数据时把噪声和随机波动也学会了，这样对新数据的预测能力甚至不如简单的线性模型。而在没见过数据上的表现能力，就是**泛化能力**。



那我们该怎么解决过拟合呢？很简单，模型太复杂了，那我们就选一个简单一点的模型。与此相对，也可以通过增加训练数据的量来解决这个问题。数据越充足，模型越不容易过拟合。

那如何获得更多的数据呢？如果没有办法直接收集更多数据，可以选择对现有的数据进行处理（图像旋转、镜像、滤镜、裁剪），就可以得到更多数据，还可以让模型的鲁棒性更强，不会因为输入的一点波动而产生很大的结果差异。

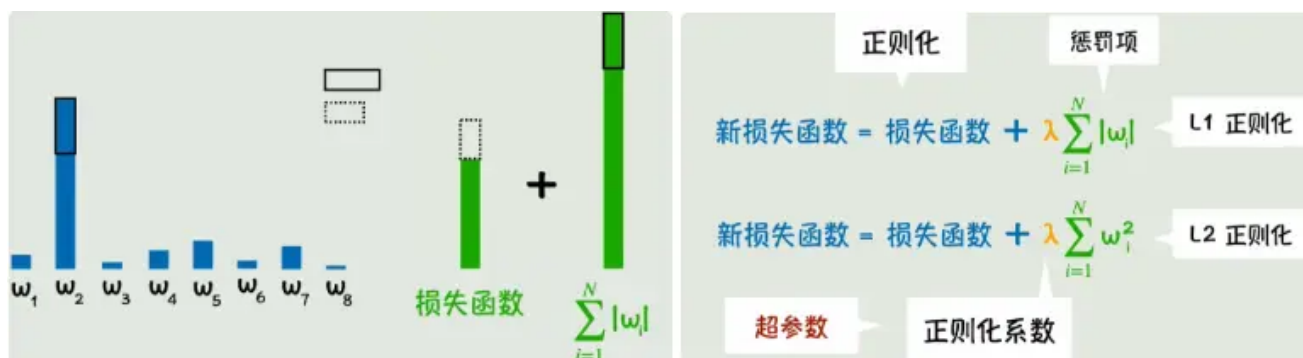
还有没有其他办法来避免过拟合呢？有的兄弟，有的。

神经网络的训练通过调整参数来让模型逼近真实数据，如果模型在向着过拟合的方向发展，那我们停止训练就好了，这样也能一定程度上避免过拟合。

但显然，这方法还是太粗糙了，有没有更精细的方法呢？有的兄弟，有的。

我们只需要在原来的损失函数的基础上加上被调整参数本身，这样当参数调整让损失函数减小的幅度甚至不如参数本身增大的幅度，新的损失函数就是增大的，这次调整显然就是不合适的（左图）。

除了加上参数本身之外，我们还可以加上参数的平方和，这样在参数大的时候，抑制的效果就更强了。我们加上的这一项就叫做**惩罚项**。把通过向损失函数中添加权重惩罚项来抑制参数野蛮增长的方法叫做**正则化**。惩罚项的力度则由**正则化系数**来控制。以上这些控制参数的参数叫做**超参数**。



除此之外，我们当然还有别的方法：Dropout

为了避免模型过于依赖某些参数，我们在每次训练时都随即丢弃掉一部分参数就好了。

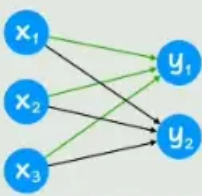
很好，现在我们了解了如何避免过拟合，那我们是否能训练出一个非常好的神经网络了呢？不，还有很多问题：

梯度消失	神经网络越深，梯度反向传播时越来越小，参数更新困难	用残差网络来防止深层网络的梯度衰减
梯度爆炸	梯度数值越来越大，参数的调整幅度失去控制	用梯度裁剪来防止梯度的更新过大 用合理的权重初始化和输入数据归一化来让梯度分布更加平滑
收敛速度慢	可能陷入局部最优或来回震荡	用动量法、RMSProp、Adam等自适应优化器来加速收敛，减少震荡
计算开销大	数据规模太大，完整的前向传播和反向传播都耗时巨大	用mini-batch把巨量数据分割成小批次，来降低单词的计算开销

对此，我们当然也有很多办法，这里简单提一下，留待补充。

矩阵表示和CNN

现在回到开始，假设我们有一个这样的神经网络，虽然参数不多，但是看起来已经很麻烦了，这时候可以考虑用矩阵来简化一下。

$$\begin{aligned}
 y_1 &= g(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) \\
 y_2 &= g(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2)
 \end{aligned}$$


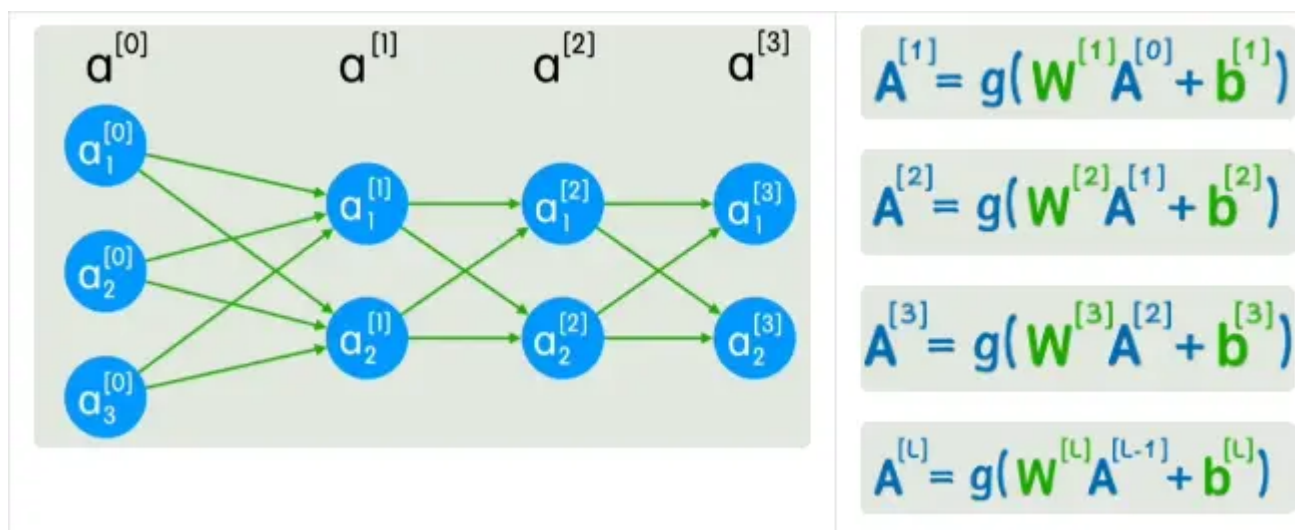
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Y
 W
 X
 b

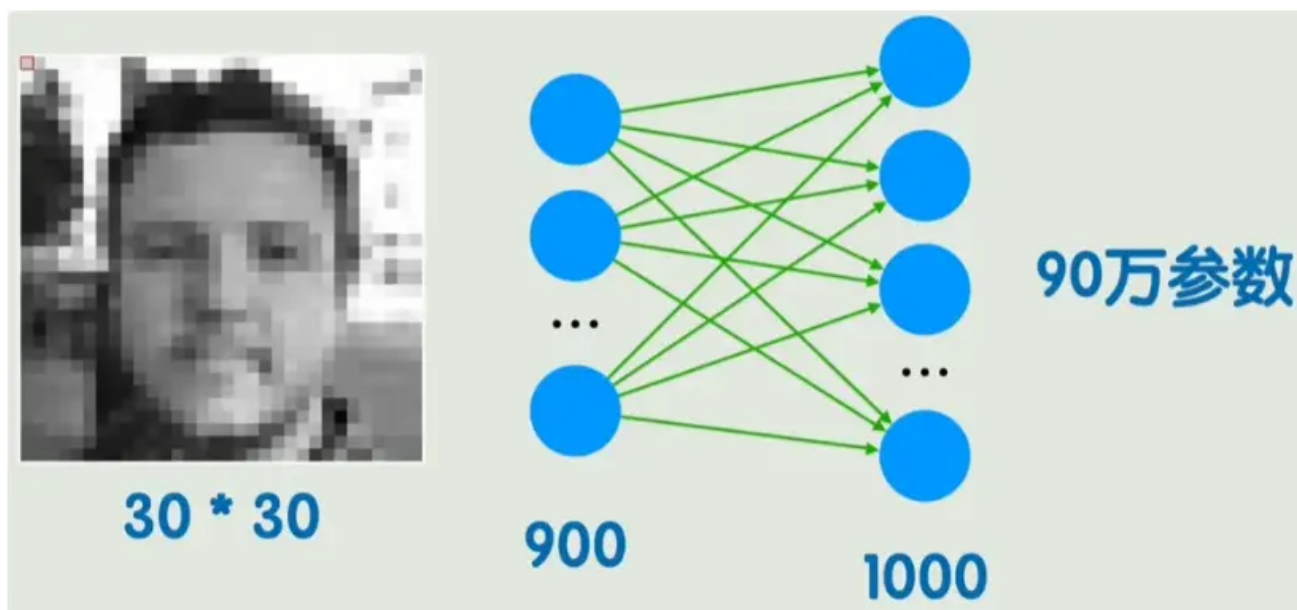
这样原本复杂的式子就可以表示为。

与此同时，当神经网络的层数越来越多的时候，也需要用合适的方法来表示。我们把输入层用 $a[0]$ 表示，中间的层就用 $a[1]$ 、 $a[2]$ 来表示，以此类推。

第一层、第二层和通式如下所示。这样不仅我们看起来更简单、更抽象了，更有利于研究更深的问题了。同时，矩阵运算相比之前的式子，可以更好的利用GPU的并行运算特性，能加速神经网络的训练和推理过程。



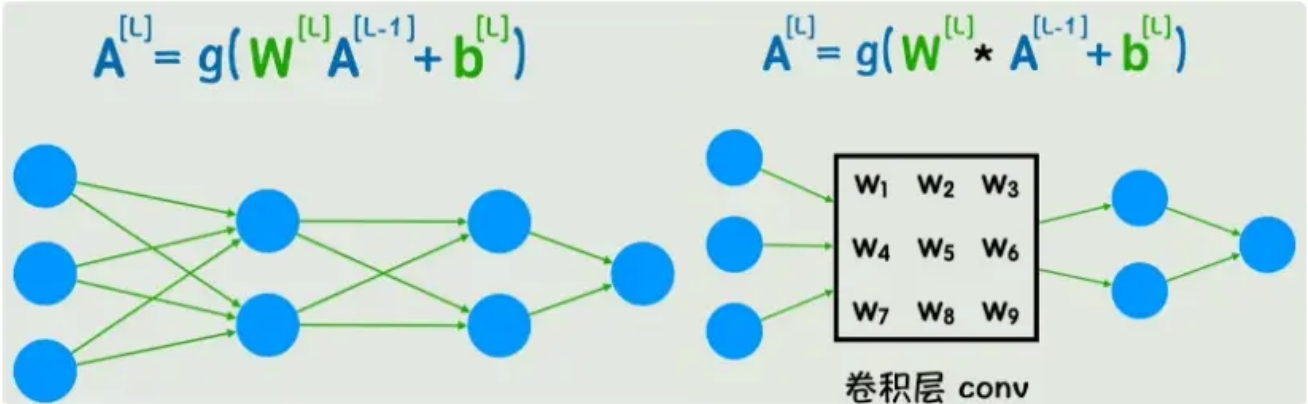
我们再来看之前的神经网络，可以发现每一个节点都和前一层的所有节点相连接，这个并非神经网络所必需的，而这种连接方式叫做全连接。全连接层有一个显而易见的缺点，对于下面的这个例子，输入900个像素，在一个全连接层之后就需要90万个参数，并且这还只是把图像平铺开，不包含每个像素之间的位置关系，如果图片稍稍平移或改变一些局部信息，但所有的神经元都会和之前不一样，这就是不能很好的理解图像的局部模式。



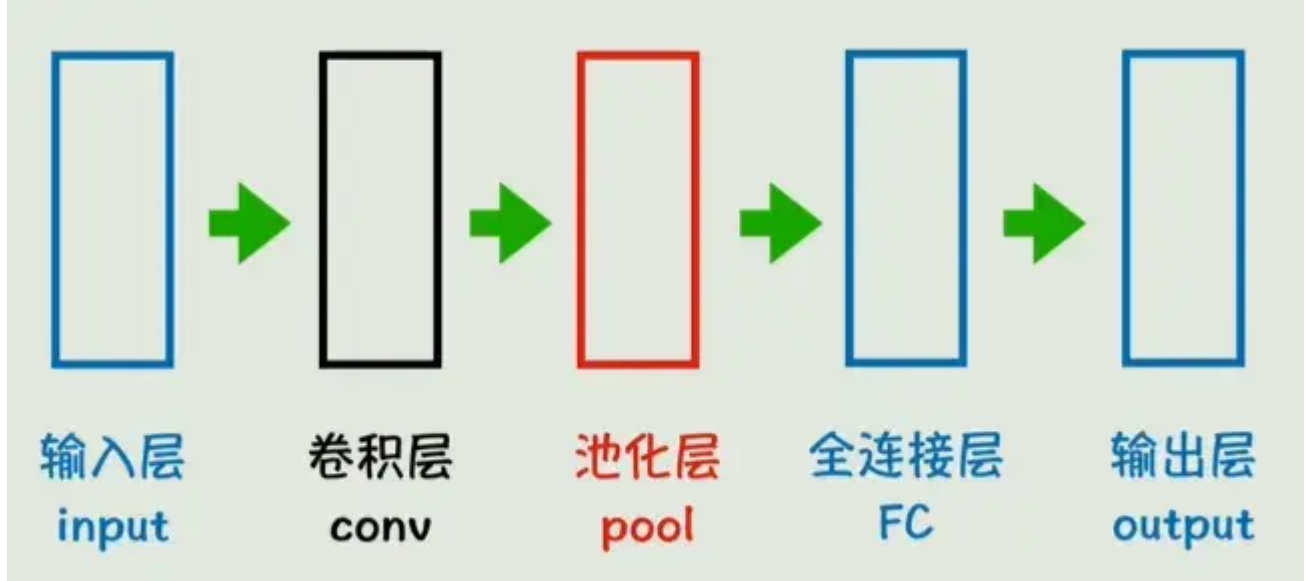
那怎么办？我们在图像中去一个 3×3 的块，将他的灰度值与另一个矩阵做运算（对应位置相乘，最后求和），遍历整张图片的所有位置，得出的数值形成一个新的图像，这种方式就叫做卷积运算。刚刚给出的矩阵就叫做卷积核。



卷积核早就被应用于传统图像处理领域，不同的卷积核可以达到不同的处理效果（轮廓、锐化、模糊）。但区别在于，图像处理中的卷积核是已知的，神经网络中我们用到的卷积核是未知的，他同样由参数构成。回到经典的神经网络结构，其实就是把一个全连接层替换为了卷积层，不仅能减少参数的数量，还能更有效的捕捉到图像中的局部信息。从公式上看，也就是把原来的矩阵标准乘法（叉乘）替换为了卷积运算。



这样我们的神经网络示意图就能简化为新的形式。可以看到多出来一个池化层，池化层的作用是降低维度的同时保留主要特征，减少计算量。图中的卷积层、池化层、全连接层都可以有多个，而这种适用于图像识别领域的神经网络结构就叫做卷积神经网络（Convolutional Neural Network，CNN）。



但是卷积神经网络依旧有它的局限性，一般来讲它只适用于处理静态数据，对于时间序列、文本、视频、音频等动态数据，就需要其他的神经网络结构了。

循环神经网络 RNN（Recurrent Neural Network）

之前的卷积神经网络适合处理图片信息，那文字信息怎么办呢？首先要明白，对于计算机，或者说神经网络来说，文字都是要转换为数字之后再进行处理。那么我们要面对的第一个问题就是：如何将文字转换为数字

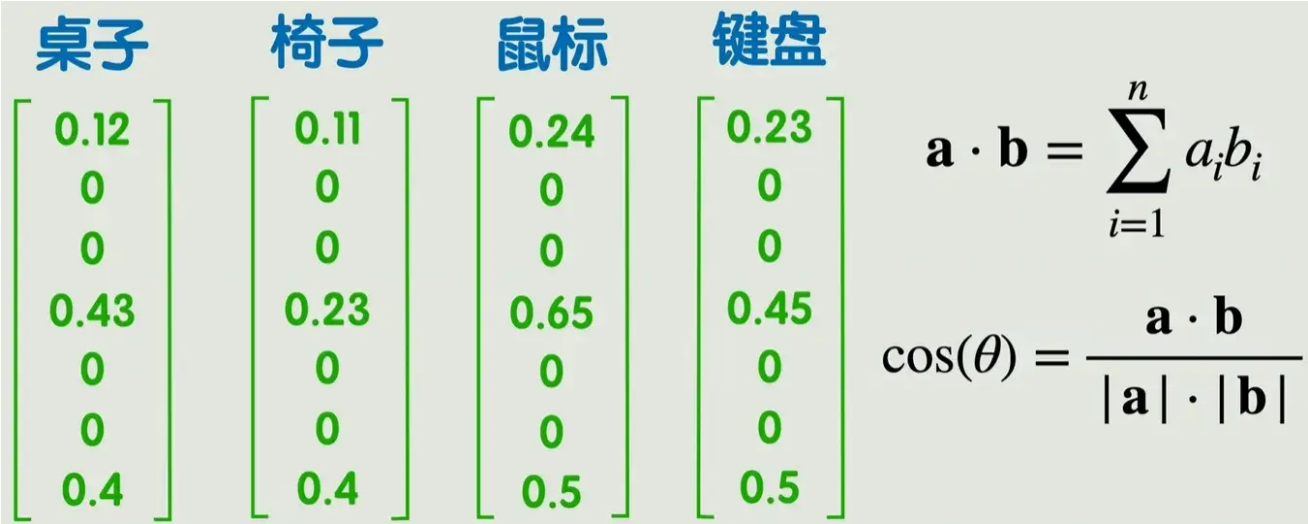
有一种简单粗暴的方法：每一个文字或词组都用一个数字来代表，建一个非常大的映射关系表

但这样有几个显而易见的缺点，第一，只用一个数字表示，不仅要建的表很大，维度也很低（只有一维），第二，数字和数字之间无法表示字与字、词与词之间的联系。为了解决维度低的问题，有人提出了one-hot编码，即准备一个维度非常高的向量，每个字只有向量中一个位置是1，其余全是0。虽然维度低的问题被解决了，但是维度好像又太高了，并且依然没有解决之前的第二个问题。

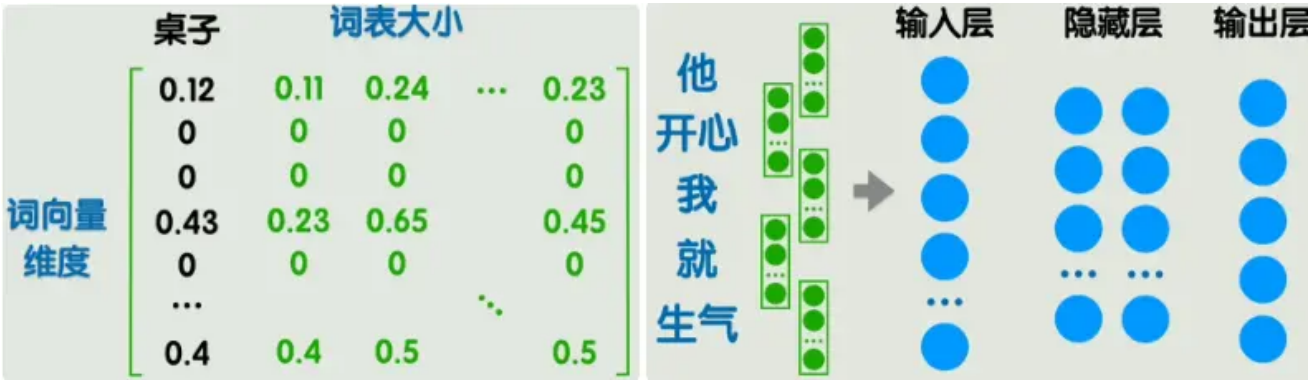
那有没有能解决以上两种问题的方法呢？有的。这种方法就是**词嵌入**。

通过词嵌入的方式得到的词向量，维度不高不低，每个位置可以理解为一个特征值，但这个特征是通过训练得到的，我们并不知道代表着什么。那这种方式如何表示词与词之间的语义相关性呢？可以用两个向量的点积或余弦相似度来表示向量之间的相关性，进而表示词语之间的相关性。

这样就将自然语言之间的联系转为可以用数学公式计算的方式。同时，一些数学上的计算结果可能反映出一些很微妙的关系，例如一个训练好的词嵌入矩阵，很可能使得桌子-椅子 = 鼠标 - 键盘。



把所有词向量组成一个大矩阵，这个大矩阵就叫做嵌入矩阵，每一列表示一个词向量。矩阵中的值由训练得到，比较经典的方法是word2vec，不展开讲解。虽然这样表示的维度比起one-hot已经大大下降，但是也超过了人能直接理解的二维、三维，我们管这些向量所在的空间叫做**潜空间**。我们无法理解潜空间中的位置关系，但是也有一些方法能够把潜空间降维至2-3维，方便我们直观看到词与词之间的关系。

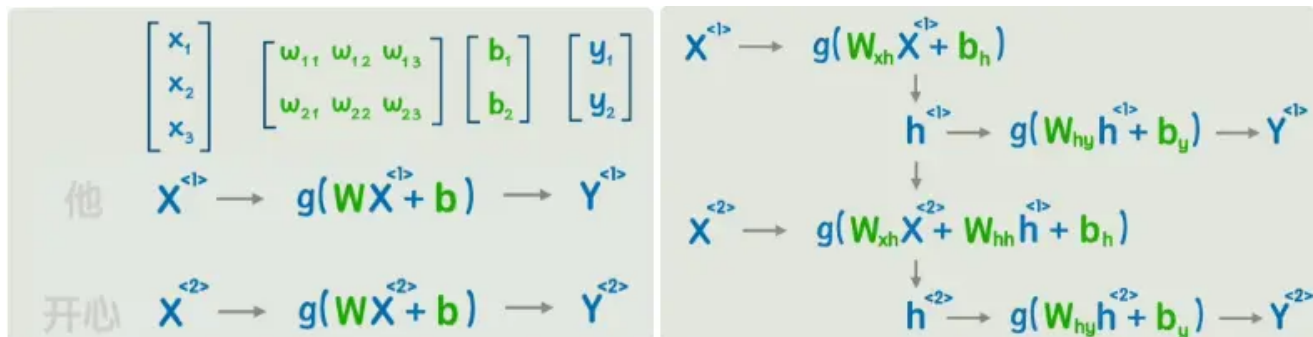


这样我们的第一个问题就算是解决了，我们可以用词嵌入的方法将文本转为数据，但这样就可以了么？举个例子，在右上方的图中，左边的五个词转为5个词向量，每个词向量假设为300维度，那么输入层就要有1500个神经元，当然是可以的，但是有两个新问题：

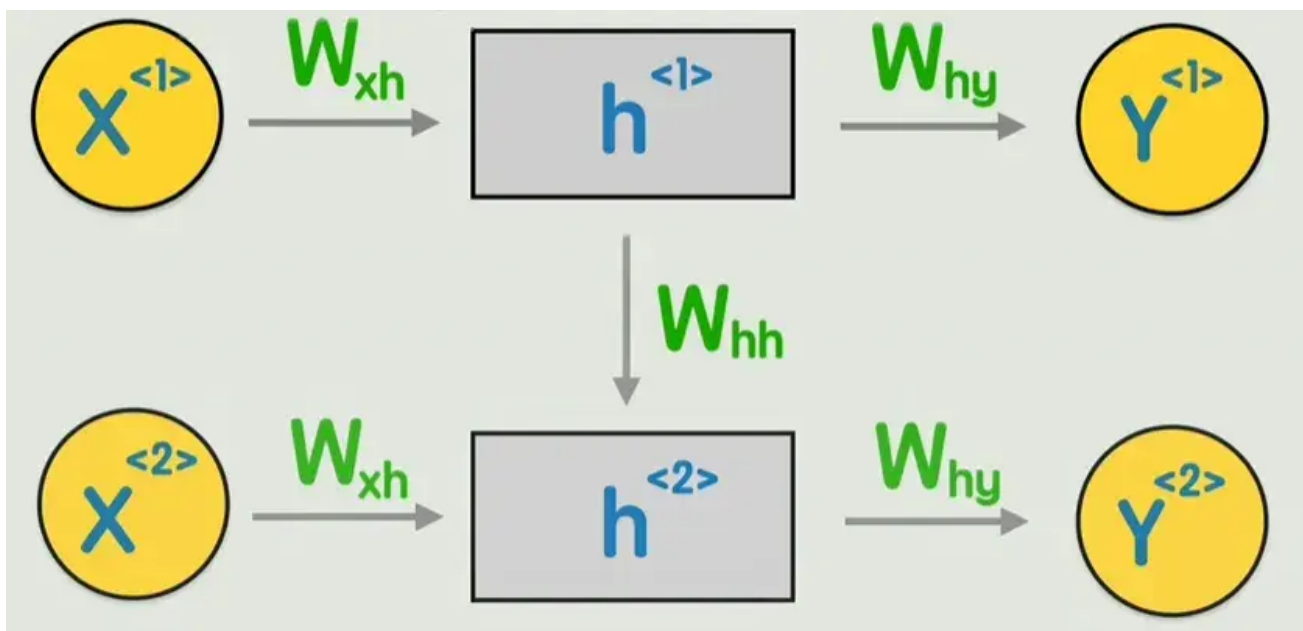
1. 输入层太大了，并且长度不固定；

2.无法体现词语的先后顺序，参考之前无法体现图片像素的位置关系。在之前的图像处理中，我们可以通过卷积的方式来解决这一问题，那现在呢？

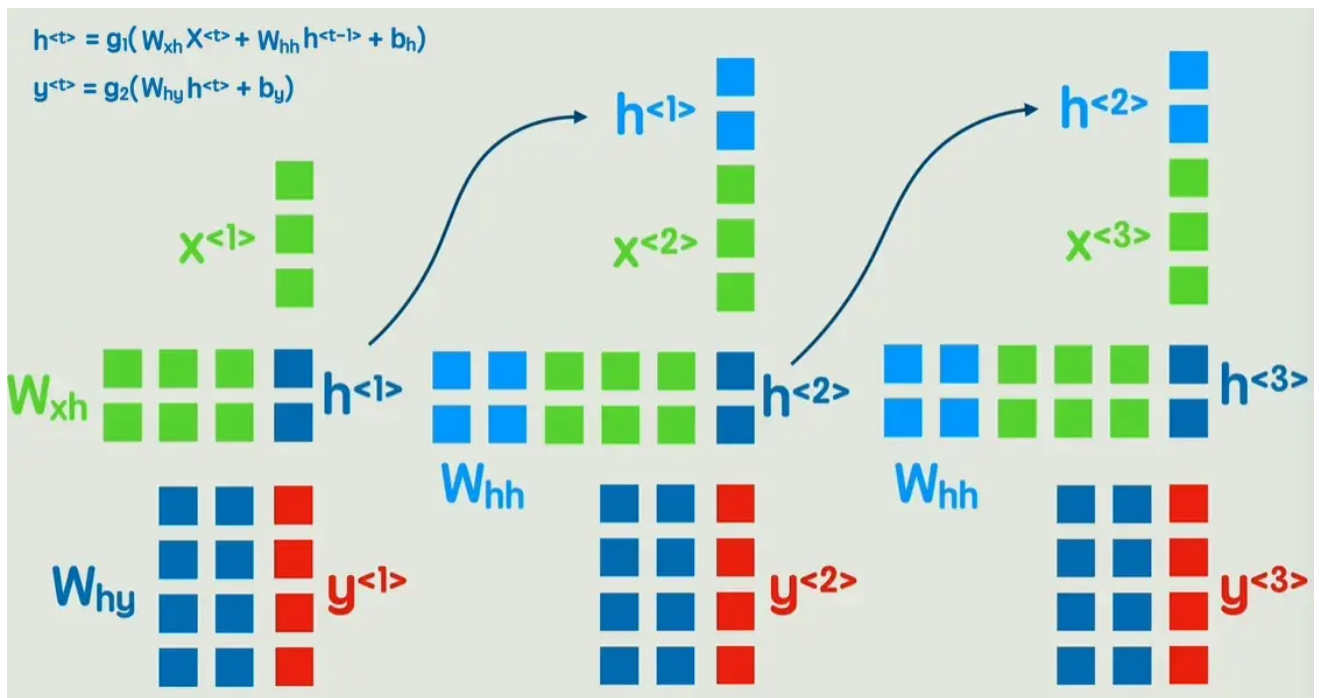
回到经典的神经网络，但是不是一次输入一句话，而是输入一个词。当然这里的X、W都是矩阵，之后不再展开。



可以发现在第二个词的计算过程中，完全没有让第一个词的任何信息参与进来，怎么办呢？可以像右图这样，先输出一个隐藏状态，然后再经过一次非线性变换，得到输出Y。这就是循环神经网络RNN。



这个RNN模型就具备了理解词与词之间先后顺序的能力，可以判断一句话中各个单词的褒贬词性，还能给出一句话，不断生成下一个字，以及完成翻译等自然语言处理工作。



那么RNN是否就完美了呢？当然不，RNN依旧存在两个问题：

- 1、信息会随着时间步的增多而逐渐丢失，无法捕捉长期依赖，而有的语句的关键信息恰好在很远的地方
- 2、RNN必须顺序处理，每个时间步必须依赖上一个时间步的隐藏状态的计算结果

虽然有一些方法在改进以上两点问题，但是还不够好，那么是否有一个可以彻底抛弃按顺序计算的新方案呢？

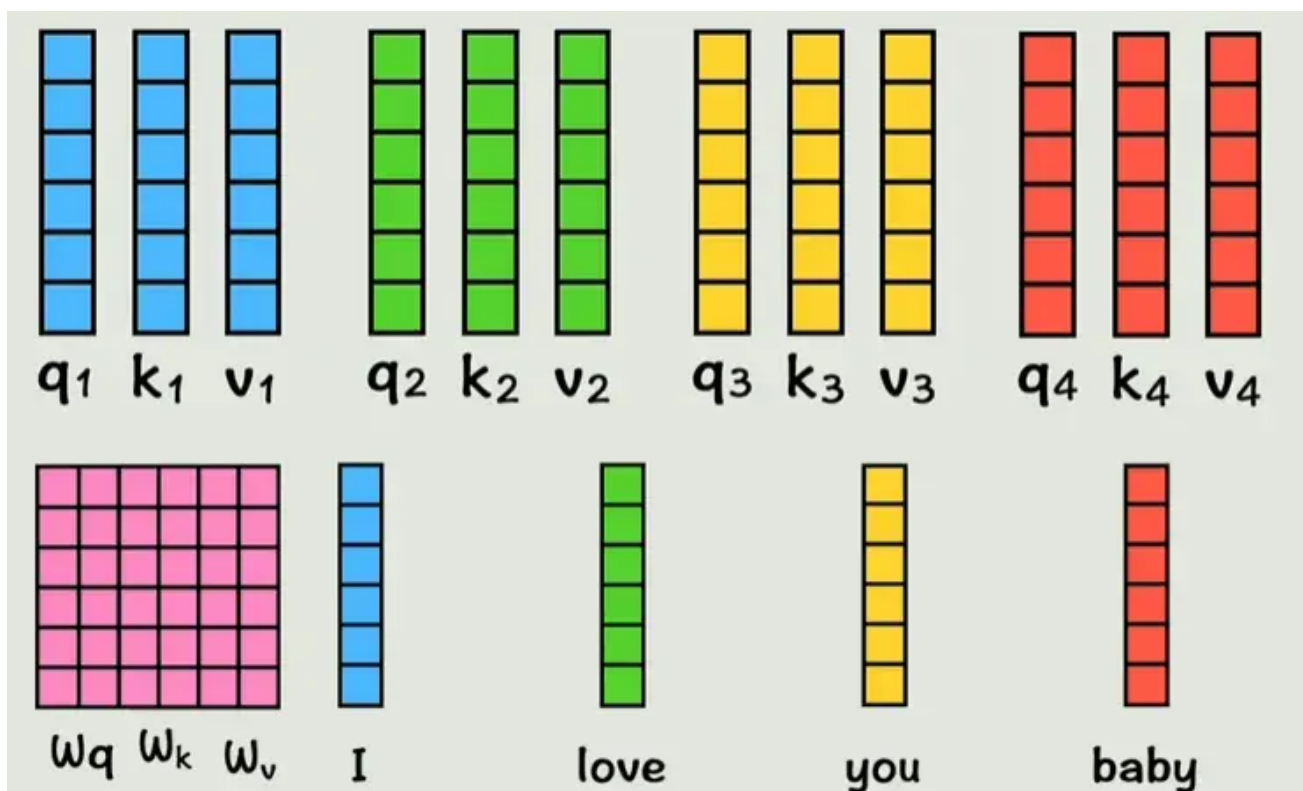
有的兄弟，有的，那就是Transformer！

Transformer架构

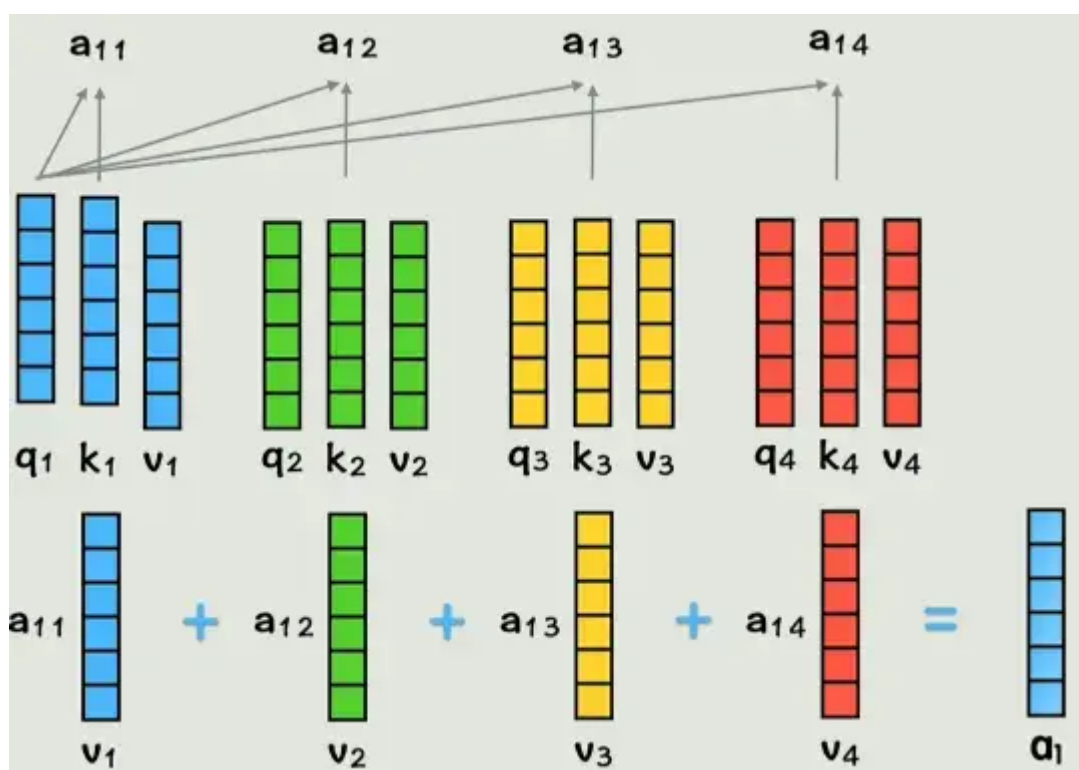
之前我们说过，对于输入神经网络的数据，要先转换为词嵌入。要解决之前提到的串行计算和长期依赖困难这两个问题，就要用到一种不同于之前的新方案--Transformer。

首先，为了让输入包含每个词之间的位置信息（前后顺序等），给每个词一个位置编码，表示这个词在整个句子中出现的位置，把这个位置编码加到原来的词向量中，现在这个词就有了位置信息。

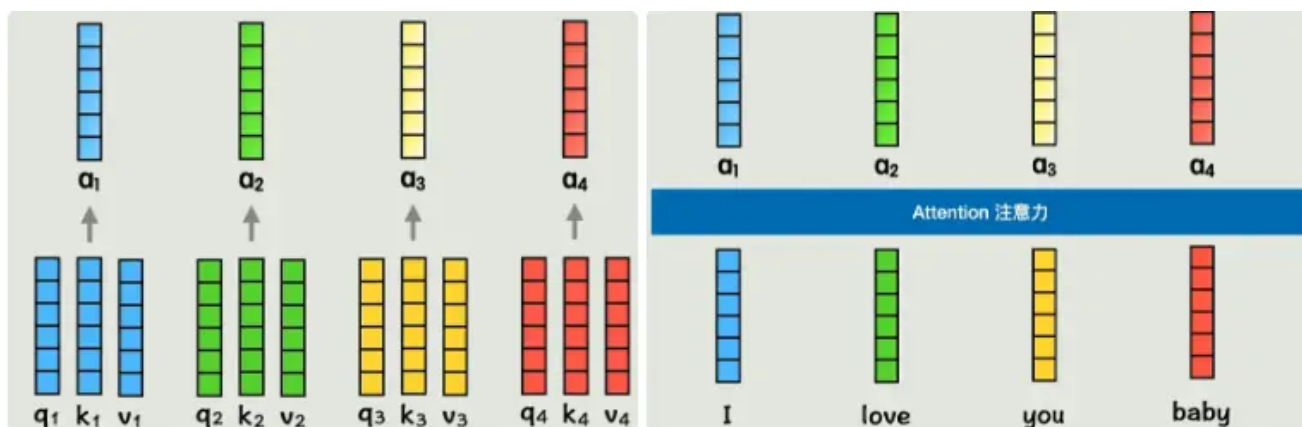
但是现在每个词中还不包括和其他词的关系，注意不到其他词的存在，所以我们用几个新矩阵 W_q 、 W_k 、 W_v （训练得到）乘上每个词的词向量。当然，在计算机中运算时，是用 W_q 、 W_k 、 W_v 直接乘下方四个词向量拼成的大矩阵，然后直接得到三个矩阵（Q、K、V）。为了方便理解，还是拆分来看。



现在我们的词向量已经通过线性变换映射为了QKV，维度不变，现在我们让 q_1 和 k_2 做点积，代表第一个词和第二个词的相似度，同理类推，得到的系数再与 v 相乘，最后相加，得到的 a_1 就是包含了全部上下文信息的第一个词的新词向量。

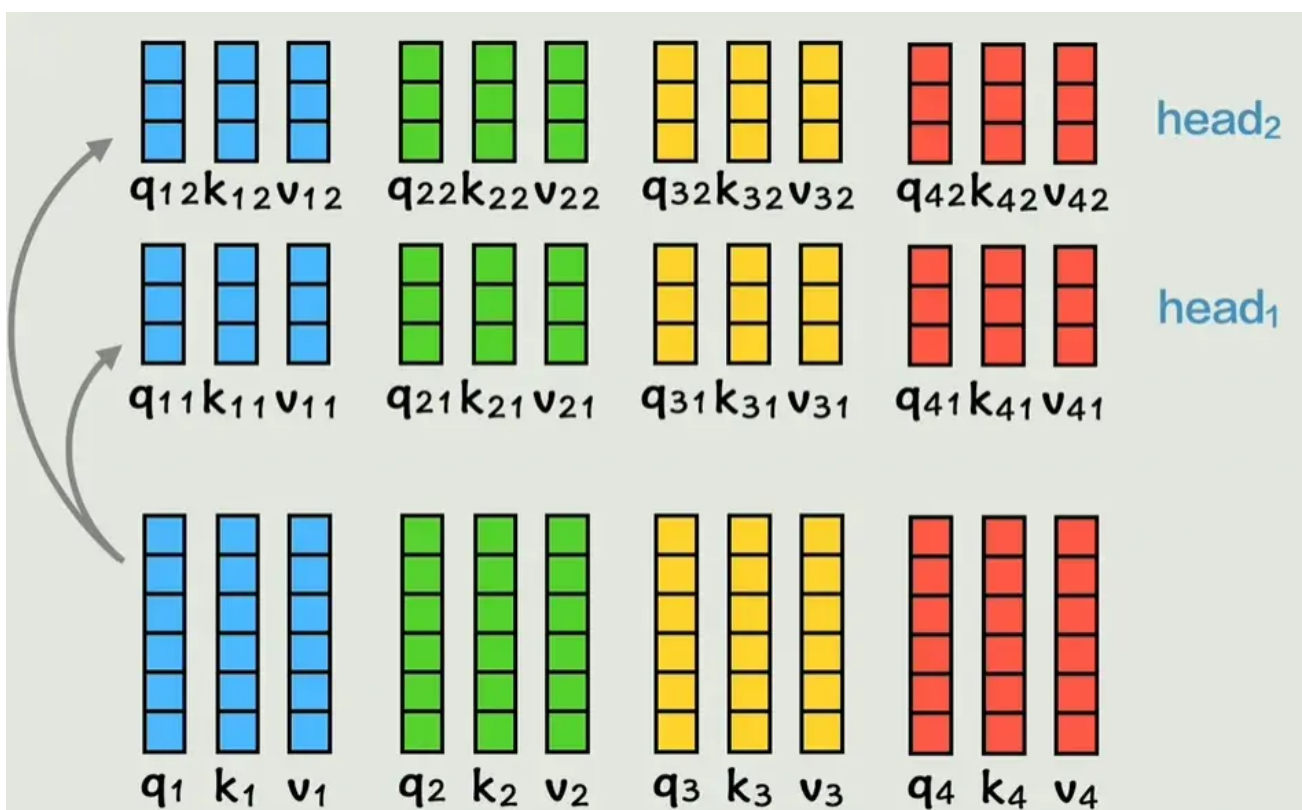


同理，我们得到了所有词的新词向量，每一个新词向量都包含了所有的上下文信息。这就是注意力机制attention所做的事情。

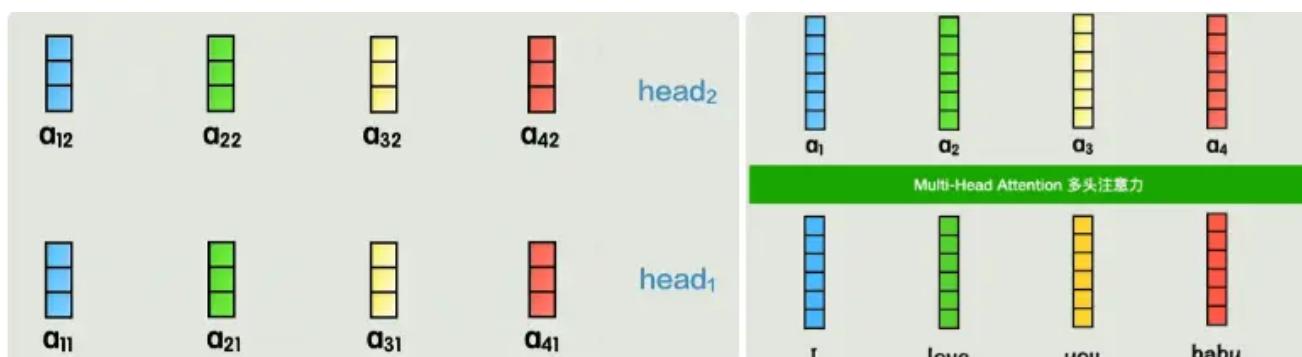


但是两个词的关系并不是固定的，对于注意力机制来说，如果只通过一种方式计算一次相关性，灵活性就太低了。

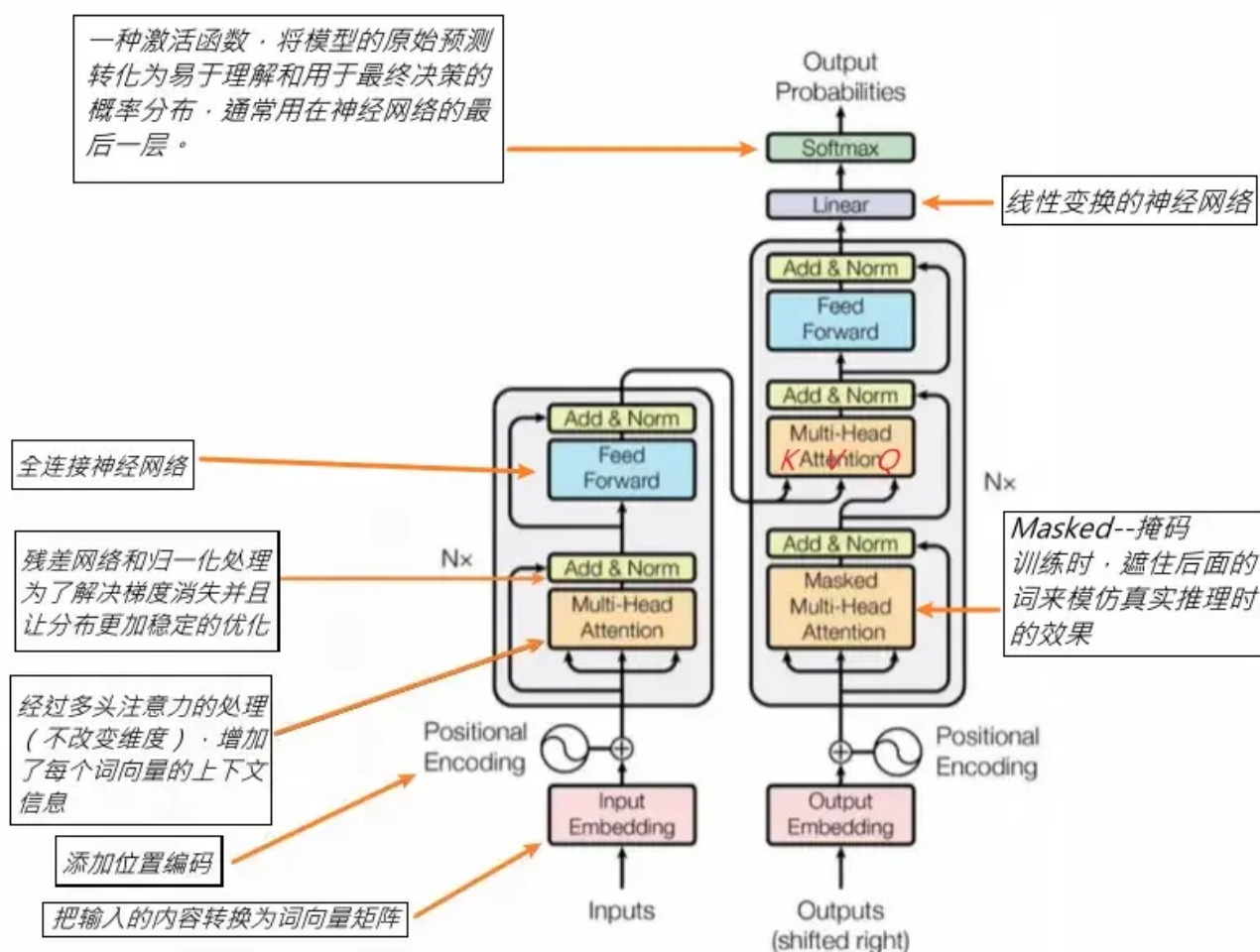
所以我们可以增加这个数量，把之前得到的QKV通过两个权重矩阵计算得到两组新的QKV，给每个词两个学习机会，每组QKV称为一个头。



再次通过之前的运算得到a向量，拼接起来就得到了和之前一样的结构。我们刚刚的例子有两个头，也属于多头注意力。

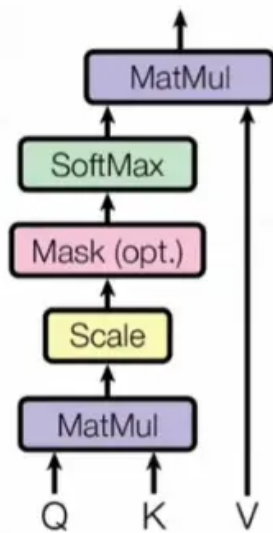


下图的左侧部分叫做编码器，右侧叫做解码器。

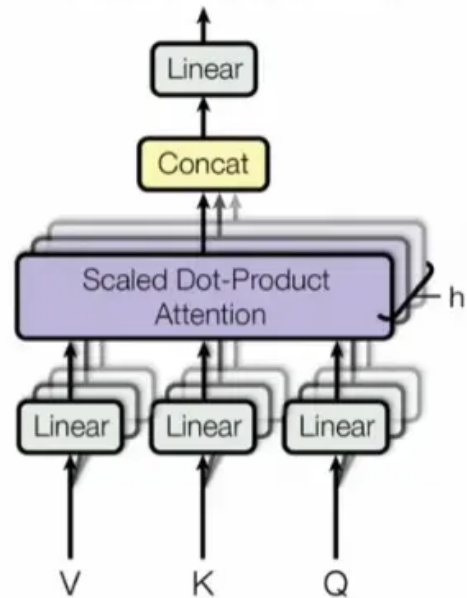


我们之前介绍的单头注意力和多头注意力都省略了一些步骤，详细的步骤和公式请看下图。

Scaled Dot-Product Attention



Multi-Head Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Over!